

Cloud Computing

Final Project

Spring 2026 · California State University San Marcos
Department of Computer Science & Engineering, College of STEM

Project & Team Information

Course	Cloud Computing – Spring 2026
Project Title	School Club Social Media Photo Re-sizer
Checkpoint Number	Checkpoint 2
Date Submitted	4/28/2026
Team Members (All)	Frankie Snedecor, Jonathan Qi
GitHub Repository URL	https://github.com/frankiesned/Social-Media-Photo-Resizer
Live Application URL (if applicable)	N/A
Video Demo	https://youtu.be/CoFrU6-w_ko Youtube
Presentation	https://docs.google.com/presentation/d/1386SjWxx_TnBxKG3SpX_4ZHuBeXFGD46ouC3OoMWkck/edit?usp=sharing Google Slides

Individual Contributions

Detail the specific tasks and contributions made by each team member for this checkpoint. Be precise—vague descriptions (e.g., “helped with backend”) will lose points.

Team Member	Role / Focus Area	Specific Tasks Completed	Approx. Hours
Jonathan Qi	Frontend Work & Frontend/Cloudend connection	- Created the html and css files for the webpage the user will be interacting with - Created a RESTful API using Flask which acts as our backend / connection to	20

		google cloud Created an input trigger which places files to the input bucket and uses Frankie's cloud function resizing and takes files from output bucket	
Frankie Snedecor	Project Direction & Cloudend work	- Coded the entire cloud function that resizes images inserted into our input bucket. - Select what format ratios we want to resize images to correspond to specific platforms. - Deployed our cloud function for resizing images with an entry point, trigger events, and output. - Created some IAM permissions for each bucket as well as for the project, for now it is broad access.	20

Table 1: Individual Contributions – include approximate hours spent per member.

1. Checkpoint Summary

1.1 Checkpoint Goal

State the specific deliverable(s) or milestone your team planned to reach by this checkpoint, as defined in your original project plan. If the goal has changed since your plan, explain why.

Action 1: Fully update the web application visually and have clear instructions on what the application is used for and how the user needs to go about it.

Johnny was able to meet this goal. The goal slightly shifted in the sense that Johnny was more focused on getting the frontend to connect with the google cloud function, and have that function return resized images back to the frontend. That goal is successful, where the user can upload an jpeg or png image and have the web page return clickable links that takes the user to the outputted resized image for easy download. Otherwise, the visual is very simplified and clean and easily readable for someone to use without need for complicated instructions.

Action 2: Implementing the media thumbnail resizing functions to crop or resize images based on what social media application is being requested

Frankie was able to meet this goal of adding the function to resize images. Johnny was able to meet his goal of implementing the website to take the images and pass them to the cloud function and display them. For cropping, both Johnny and Frankie have decided to push off on that option for now as we polish the project, and given we have time we can implement it. For now, the cloud function takes in a single image and resizes it into 10 different formats corresponding to different social media platforms.

Action 3: Integrating the web application to access our google cloud application to access their buckets for fully functional use.

This goal has been met, currently our project functions as follows: After running our server python app, the user loads the website and the user inserts an image on the website. Upon upload, the image is passed to the initial cloud bucket. Our cloud function is set to trigger upon upload of image into the initial cloud bucket. So upon uploading an image our cloud function is run, resizing the image to 10 different ratios, and sending those buckets into a resize output bucket. The website then grabs the corresponding images from the output bucket and displays the links to the resized images so they aren't all cluttered on the page.

1.2 Deliverables Completed

List each completed task with enough detail that the instructor can verify it during the demo. Include the cloud service(s) involved where applicable.

#	Task / Feature	Cloud Service(s) Used	Evidence / How to Verify
1	User can upload a jpeg or png image from their personal computer and have it uploaded to the input bucket in Google Cloud Storage	Google Cloud Storage	Verify by uploading an image and checking google cloud console, specifically the input storage bucket for the jpeg or png file
2	User can view the multiple resized images of originally inputted image on webpage after waiting a couple	Google Cloud Storage	Verify by checking google cloud console, specifically the resized storage bucket for

	seconds after inputting		multiple resized files OR check webpage to see the links pop up
3	Input images, png or jpg, can be resized to 10 different format sizes keeping the ratio of the image to prevent squeezing or stretching.	Google Cloud Function	Verify by viewing the output bucket with the resized images and verify by checking the image properties to verify the pixel size has been changed
4	When deploying our google cloud function, it includes a trigger upon upload to the initial bucket. This runs the code segment in our main.py that runs our resizing algorithm. Outputs are sent to an output bucket.	Google Cloud Function	Verify by checking output bucket AFTER putting something in the initial bucket, if output bucket receives new files of the inputted image, that means the trigger AND the resizing function worked

Table 2: Completed Deliverables

1.3 Planned Deliverables Not Completed

For each incomplete task, explain the root cause and your concrete remediation plan. "Ran out of time" is not an acceptable root cause, dig deeper.

#	Incomplete Task	Root Cause Analysis	Current Status (%)	Remediation Plan
1	The ability to crop images to specific sizes.	We wanted to polish what we have instead of implementing a lot of features off the start. We want to discuss the feasibility of cropping vs resizing.	50%	I would say 50% as our initial goal was to have both resizing and cropping, but we ultimately dropped it to focus on other aspects. Once polished it would be an easy addition for our final submission.

2	Categorization of links of output images.	We spent more time focusing on getting the frontend and backend to connect. For now we just have a simple display of links for each image for testing.	75%	It will be pretty easy to ca
---	---	--	-----	------------------------------

Table 3: Incomplete Deliverables – include % completion and remediation.

2. Demonstration Artifacts (Mandatory)

Both artifacts below are required. Missing either one will result in a grade penalty.

2.1 Video Demonstration (5–10 minutes)

Provide a link to a video recording demonstrating your project's current state. The video must be set to Unlisted or Public on YouTube (or equivalent shareable link) for grading.

Required content:

1. Brief introduction from each team member (name, role).
2. Clear walkthrough of the user-facing application (if any).
3. Live demonstration of every feature completed for this checkpoint.
4. Tour of the cloud infrastructure backend (console view showing active services, configurations, deployed resources).
5. Walkthrough of the code repository structure and key modules.
6. Discussion of one significant technical challenge you solved and how.

YouTube / Shareable Video Link: https://youtu.be/CoFrU6-w_ko

2.2 Checkpoint Presentation Slides (5–8 slides)

Provide a link to a brief slide deck summarizing your progress.

Required slides:

1. Title Slide – Project name, team members, checkpoint number.
2. Project Goal & Scope – One-sentence project summary and target users.
3. Updated Architecture Diagram – Must reflect the *current* state of your system.
4. Work Completed This Checkpoint – Demo highlights, key screenshots.
5. Challenges Faced & Lessons Learned.
6. Plan for Next Checkpoint – Specific tasks with owners and deadlines.
7. Cost Report – Cloud resource costs incurred to date (even if \$0 under free tier).

Google Slides / PowerPoint Online Link:

https://docs.google.com/presentation/d/1386SjWxx_TnBxKG3SpX_4ZHuBeXFGD46ouC3OoMWkck/edit?usp=sharing

3. Technical Review

3.1 Cloud Architecture & Services

Provider & Region

Cloud Provider(s) and Region(s): GCP & GCS us-west1

Services in Use

Complete the table below for every cloud service your project currently uses.

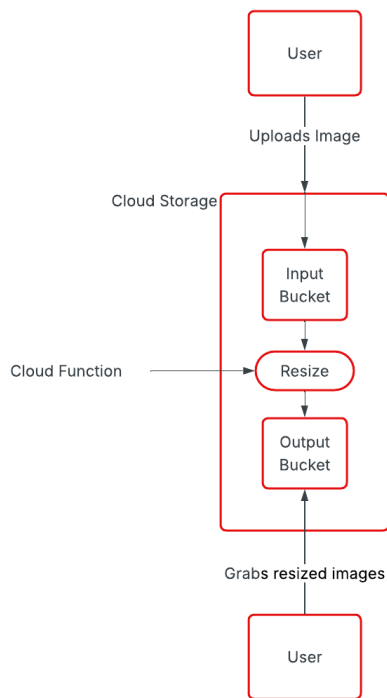
Service Name	Category	Purpose in Your Project	Configuration Notes
GC Functions	Compute	Resize images. Function is triggered upon image upload to input bucket, and sends resized images to output bucket.	Can take in png, jpg, and jpeg, outputs are only jpegs.
GCS buckets	Storage	Input bucket to trigger cloud function upon image upload and output bucket to store resized images.	Set to delete objects inside after 1 day.
GCS Buckets	Database	We don't need to store photos, they are only used for the user one time. Buckets provide a cheaper and efficient alternative.	Buckets are set to store images, and delete images after a certain amount of time.
N/A	Networking	Eventually we will implement the ability to have a public webpage, as shown in the not reached deliverables section. Currently we are running the program locally.	N/A
GCP	IAM / Security	• The initial bucket has objectViewer permissions. • The output bucket has objectAdmin permissions. • The project has pubsub publisher role	As we finalize our project we will correctly set permissions to the least access possible.

Table 4: Cloud Services Inventory

Architecture Diagram

Architecture Diagram Link: (Must be current. Attach as an image or provide a shareable link: e.g., draw.io, Lucidchart, Excalidraw.)

Describe any changes to your architecture since the last checkpoint and the reasoning behind each change



Our previous design had multiple input buckets, cloud functions and output buckets. After researching and implementing we realized that we would only need one of each. This is because the input bucket can hold multiple images, and the output bucket can hold multiple images as well. This current design shows the process that users can both upload images and grab resized images. Once the image is uploaded to the input bucket, the cloud function is triggered which then sends it to the output bucket.

3.2 API & Data Model

API Endpoints

List all new or updated endpoints. Include the HTTP method, path, brief description, and authentication requirement.

Method	Endpoint Path	Description	Auth Required?
POST	/upload	Takes user inputted image and uploads it to the input storage bucket in Google Cloud Storage	No (Not yet implemented)
GET	/list	Retrieves new file names from output bucket in Google Cloud Storage after resizing	No (Not yet implemented)

Table 5: API Endpoint Inventory

Data Model / Database Schema

Describe your current database schema, any new tables or collections added this checkpoint, and relationships between entities. If you have an ERD, provide a link:

For our specific project, we do not have a conventional database or database schema, we are utilizing buckets as our “database” since that holds all our images and the functions required to resize them. For further details about our pipeline please refer to the diagram above the **Architecture Diagram** section

3.3 Security & Compliance

Security Measures Implemented (e.g., IAM roles/policies, API key management, authentication/authorization mechanism, VPC configuration, encryption at rest/in transit, secrets management):

gsutil iam ch
serviceAccount:service-10189932669@gcp-sa-eventarc.iam.gserviceaccount.com:objectViewer
gs://media_image_initial_bucket

Object Viewer permission allows Eventarc to be able to read files from our input bucket. This is used to detect when a file is uploaded so it can trigger our cloud function.

gsutil iam ch
serviceAccount:service-10189932669@gcp-sa-eventarc.iam.gserviceaccount.com:objectAdmin
gs://media_image_resized_bucket

Object Admin permission allows Eventarc to be able to upload, overwrite, delete files for our output bucket. This allows the cloud function to write to our output bucket as it needs to upload the files to it.

gcloud projects add-iam-policy-binding social-media-photo-resizer
--member="serviceAccount:service-10189932669@gs-project-accounts.iam.gserviceaccount.com"
--role="roles/pubsub.publisher"

This is a role for our entire project that allows event messages to be sent to the trigger function.

Testing Strategy & Progress (e.g., unit tests written/passing, integration tests, load testing, manual QA results, CI/CD pipeline status):

In terms of testing we would mostly focus on the activity within our buckets to determine if the resizing function would work. For example once we input an image to the initial bucket, we would check the resized bucket to see if the resized images would output successfully.

Other testing methods were more focused on the webpage and frontend application. First we had to test whether or not our web page can successfully take an image file. After that the testing came to whether or not my API was successfully putting that image into the google cloud storage bucket. This was all assuming the actual resizing function was working once put into the bucket. We just wanted to focus on whether it was successfully connecting to those buckets and retrieving them from the resized bucket.

We would also test multiple different file types to make sure it would only work with the main image files such as jpeg and png files.

3.4 Cost Management

Total Cost to Date	\$0.003259
Projected Cost (End of Semester)	\$0.01
Free Tier Services Used	Our free \$300 credit ran out since it was past the 90 day mark. We are currently using a \$50 educational credit given by our professor.
Cost Optimization Steps Taken	Using buckets instead of a database. Currently our project runs locally instead of publicly.

Attach a screenshot of your billing dashboard (AWS Cost Explorer, GCP Billing, or Azure Cost Management) in your presentation slides.

Billing Account: Billing Account for Education (Account ID: 01FF04-E70A43-A0EDA8)
Time range: April 2026
Time granularity:
Group by Products: All services (8)
All subaccount, SKU, and label are selected.

Current month, Group by Service (edited)

Save as new

Reset filters

April 1 – 27, 2026 (total cost)

\$0.00

includes \$0.00 in savings

–

\$0.00 over March 5 – 31, 2026

April 1 – 30, 2026 (forecasted total cost)

Not enough historical data to project cost



Download CSV

Service	Usage cost	Savings programs	Other savings	Subtotal	% Change
Cloud Run Functions	\$0.00	–	–	\$0.00	New
Cloud Build	\$0.00	–	–	\$0.00	New

Subtotal \$0.00
Tax –
Filtered total \$0.00

Included projects (1) 10189932669

3.5 Challenges and Blockers

Describe the technical or non-technical difficulties your team faced at this checkpoint. For each challenge, explain: (1) what the problem was, (2) what you tried, and (3) how you resolved it, or what your plan is if it remains unresolved. Were there any blockers (e.g., permissions issues, service limits, API bugs, team coordination problems)?

Technical difficulties included

Finding a way to connect the front end and have it connect to our google cloud storage through a backend. We tried considering making a Dockerfile to make the google client run continuously and try that way, but we wanted to try to get it working locally. So with research, we ended up using RESTful APIs through Flask to authenticate ourselves to be able to access the google cloud project, which after that allows for the frontend to input the selected file.

Another technical issue is that when the function WAS working, the output images were originally not displaying correctly, mostly showing 404 or 403 not found. This was because of the formatting I was expecting the outputted files to be. For example the outputted images will always have the image name, NEW, and the pixel size. There was some really specific spacing or period placement that I was messing up and it took forever to figure it out, more than it really should. But that took a significant amount of time so it deserves a spot in this blocker section. This ended up getting scrapped because we opted to instead show links and not the images themselves.

4. Code Quality & DevOps

4.1 Repository Structure

Primary Repository URL: <https://github.com/frankiesned/Social-Media-Photo-Resizer>

Describe your repository structure and branching strategy (e.g., main/dev/feature branches, PR review process):

For our repository structure, we decided to separate the cloud function and the website. We created two folders, Cloud Media Resizer and Web Media Resizer. This is to keep the code more clean and display a separation between frontend and backend. The Cloud Media Resizer folder contains the files for the cloud function and buckets to function. The Web Media Resizer folder contains the files for the website to work and connect the bridge between the frontend and backend.

For branching, we kept it simple by deploying all our changes into the main branch instead of creating separate ones. Once we get the project finished, we may implement a test branch to implement drastic changes, like new features or implementing large changes or bug fixes.

4.2 Continuous Integration & Continuous Deployment

CI/CD Tool	(e.g., GitHub Actions, Jenkins, Cloud Build, Code Pipeline) Github Actions and Cloud Build
Automated Tests?	☐ Yes ☐ No ☐ In Progress No

Deployment Method	(e.g., manual, automated via pipeline, IaC) Manual
Infrastructure as Code?	☐ Terraform ☐ CloudFormation ☐ Pulumi ☐ None None
Live Endpoint URL	None

5. Retrospective & Planning

5.1 Checkpoint Retrospective

What Went Well

Identify specific successes, technical wins, effective team practices, or process improvements:

One of our major successes as a duo we would say is constant communication. Once we set concrete goals for this sprint, we all have trust in each other to get it done by the specified time. With these team practices, we were successful in our project deliverables for this sprint. We successfully implemented the cloud functions required AND made a frontend that can retrieve and respond to a backend which connects to the google cloud functions.

What Could Be Improved

Be honest and specific. What processes, communication patterns, or technical approaches need to change?

We would say that we could spend more time allocating time to get together and code together. There were a couple of times where one of us would have to test our code by using the other's code as a dependency and there would be a small error. But since we would work separately, it would take time to debug or it would take time to ask the other person to explain or fix the bug for us. Knowing this, moving forward we would take time before the due date BUT after we "finished our code" to get together and try to work on our code simultaneously to ensure connection is good between front end and backend

Key Lessons Learned

What is the single most important thing your team learned at this checkpoint that will change how you work going forward?

Take however long you think your task is going to take, and assume it will be at least 1.5x longer. Especially when you're dealing with something new, there are always going to be roadblocks that you are going to have to handle, which will take you longer. Another thing is starting small, and expanding after the code is working and cleaned up. Trying to implement lots of features without a framework can lead to more time spent fixing bugs and changing options when features don't end up working correctly.

5.2 Action Items for Next Checkpoint

List concrete, measurable tasks with assigned owners and deadlines. Each task should be specific enough that completion can be objectively verified.

#	Task Description	Owner	Deadline	Definition of Done
1	Successfully categorize output image links with what social media platform they are designated to	Frankie/Johnny	May 7th	Web page outputs categorized social media
2	Ability to crop images to specific pixel lengths.	Frankie	May 7th	Could add a secondary bucket, or just fill a regular output bucket.
3	Make the program website run on a live server.	Johnny	May 7th	For our final deliverable, it should be accessible from anyone.

Table 6: Action Items – must include owner, deadline, and definition of done.

5.3 Risk Analysis & Mitigation

Identify the top risks that could prevent your team from succeeding in the next checkpoint.

#	Risk Description	Likelihood	Impact	Mitigation Plan
1	User could accidentally upload a file that exposes something super private	☐ H ☒ M ☐ L	☐ H ☐ M ☒ L	Our system has a midway point, where we have two separate buttons between the user uploading, and for the project to start resizing, so the user can preview their image before processing.
2	A user can upload an unlimited amount of images. This can cause large expenses and cause crashes.	☒ H ☐ M ☐ L	☒ H ☐ M ☐ L	Give each user a specific amount of uploads they can make.

Table 7: Risk Register – rate likelihood and impact as High / Medium / Low.

6. Team Self-Assessment

Rate your team's performance on the following dimensions for this checkpoint.

Dimension	1	2	3	4	5
Technical Execution	☐	☐	☐	☒	☐
Cloud Best Practices	☐	☐	☐	☒	☐
Team Communication	☐	☐	☐	☐	☒
Task Distribution / Equity	☐	☐	☐	☐	☒
Code Quality & Documenta tion	☐	☐	☐	☒	☐
Time Management	☐	☐	☐	☐	☒
Progress vs. Plan	☐	☐	☐	☐	☒

Scale: 1 = Significantly Below Expectations 5 = Exceeds Expectations

Brief justification for your ratings (1–2 sentences per low-scoring dimension):

Because we are still fairly new with applying our cloud computing practices in real projects, we aren't 100 percent sure the techniques we are using are the most efficient for the goal we are trying to reach. But, we are using practices that we are able to understand and it does work, so that's why we didn't write any really low scoring ratings.

Instructor Feedback

(Reserved for instructor use.)